

Aula 14: Redes Neurais em Grafos (GNNs)

De Difusão em Grafos a Arquiteturas de Message Passing

Eliezer de Souza da Silva

Mestrado e Doutorado em Ciência da Computação (MDCC)
Universidade Federal do Ceará

Novembro de 2025

Roteiro

- 1 Fundamentos de Grafos
- 2 Tarefas e Algoritmos Clássicos
- 3 A Receita: Message Passing
- 4 Expressividade e Limites (WL Test)
- 5 GNNs e Transformers
- 6 Encerramento

Para Além de Grades e Sequências

Até agora, as nossas arquiteturas de deep learning têm sido especializadas para dados com uma estrutura muito regular:

- **CNNs (Grades):** Excelentes para dados com topologia em grade (imagens). O kernel 3×3 explora a vizinhança local.
- **RNNs / Transformers (Sequências):** Excelentes para dados sequenciais (texto, séries temporais). A recorrência ou a auto-atenção exploram a ordem temporal.

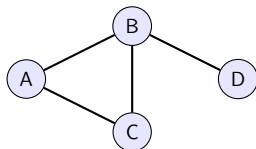
A Questão

E se os dados não forem uma grade nem uma sequência? E se a estrutura for **irregular e relacional**?

O Mundo é Relacional

Exemplos:

- Redes Sociais
- Moléculas (fármacos)
- Redes de Citação
- Internet (Links)
- Redes de Proteínas
- Sistemas de Recomendação



Precisamos de uma rede neural que opere diretamente em Grafos.

O que é um Grafo? Definição Formal

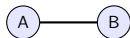
Definição

Um grafo G é um par $G = (V, E)$, onde:

- $V = \{v_1, \dots, v_N\}$ é o conjunto de N **nós** (ou vértices).
- $E \subseteq V \times V$ é o conjunto de **arestas** (ou ligações).

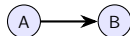
Grafo Não-Direcionado

- As arestas são pares não ordenados.
- $(u, v) \in E \implies (v, u) \in E$.
- Ex: Amizade no Facebook.



Grafo Direcionado (Digrafo)

- As arestas são pares ordenados.
- $(u, v) \in E$ *não* implica $(v, u) \in E$.
- Ex: Segue no Twitter.



Representando Grafos: Features I

Um grafo não é apenas a sua topologia. Os nós e arestas transportam informação.

Features de Nós (Node Features) X_V

Uma matriz $X \in \mathbb{R}^{N \times F_N}$, onde $N = |V|$ é o número de nós e F_N é a dimensão das features de cada nó.

- **Molécula:** X_i (para o átomo i) pode ser [tipo de átomo (one-hot), carga elétrica, massa atômica, ...].
- **Rede Social:** X_i (para o utilizador i) pode ser [idade, n.º seguidores, média de likes, vetor de embedding da foto de perfil, ...].

Representando Grafos: Features II

Features de Arestas (Edge Features) X_E

Uma representação para as arestas, $E_{uv} \in \mathbb{R}^{F_E}$, onde F_E é a dimensão das features de cada aresta.

- **Molécula:** E_{uv} (para a ligação $u - v$) pode ser [tipo de ligação (simples, dupla), distância, ...].
- **Rede Social:** E_{uv} (para a amizade $u - v$) pode ser [data da amizade, n.º de mensagens trocadas, ...].

Objetivo da GNN

Uma GNN é uma função $f(A, X_V, X_E)$ que aprende com toda esta informação.

O Laplaciano do Grafo: Medindo “Suavidade” I

Para os algoritmos, representamos a topologia do grafo usando matrizes.

- **Matriz de Adjacência (A):** $A \in \mathbb{R}^{N \times N}$. A entrada $A_{ij} = 1$ se houver uma aresta do nó i para o nó j , e 0 caso contrário. (Para arestas com pesos, $A_{ij} = w_{ij}$).
- **Matriz de Grau (D):** $D \in \mathbb{R}^{N \times N}$. Uma matriz diagonal onde D_{ii} é o número de arestas ligadas ao nó i (o seu grau).
- **Matriz Laplaciana (L):** $L = D - A$. Esta é a matriz mais importante na Teoria Espectral de Grafos.

O Laplaciano do Grafo: Medindo “Suavidade” II

O Laplaciano mede a suavidade de um sinal (um vetor de features x) no grafo. A operação Lx é um operador de diferença, análogo à segunda derivada (∇^2) no cálculo contínuo.

$$(Lx)_i = (D - A)x_i = \sum_{j \in \mathcal{N}(i)} (x_i - x_j)$$

Se o valor de x_i for muito diferente dos seus vizinhos, o resultado $(Lx)_i$ será alto.

O Laplaciano do Grafo: Medindo “Suavidade” III

O que L mede? (Intuição)

O Laplaciano é um **operador de diferença** (análogo a uma derivada). Aplicá-lo a um vetor de features de nós x (onde x_i é o valor do nó i) mede o quão suave o sinal x é no grafo.

$$(Lx)_i = \sum_{j \in \mathcal{N}(i)} (x_i - x_j)$$

- Se um nó i tem um valor x_i muito *semelhante* aos seus vizinhos x_j , o resultado $(Lx)_i$ será próximo de zero.
- Se x_i for muito *diferente* dos seus vizinhos, $(Lx)_i$ será grande (positivo ou negativo).

A energia total do sinal no grafo é $x^T Lx = \sum_{(i,j) \in E} (x_i - x_j)^2$. Um valor baixo significa que o grafo é suave.

Intuição: Difusão de Informação

Muitos algoritmos clássicos de grafos baseiam-se numa ideia central: **difusão de informação**.

A Homofilia em Grafos

Um nó é definido pelos seus vizinhos.

- **Redes Sociais:** Você provavelmente partilha interesses com os seus amigos.
- **Citações:** Um artigo está provavelmente relacionado com os artigos que cita e que o citam.

Algoritmos clássicos formalizam isto como um processo iterativo onde os nós trocam informação com os seus vizinhos até um estado de equilíbrio (convergência).

Difusão e Cascades em Grafos I

A ideia de que um nó é definido pelos seus vizinhos pode ser formalizada como um processo de difusão.

Difusão e Cascades em Grafos II

Difusão de Informação (Heat Diffusion)

O Laplaciano L descreve como a informação (ex: calor) se espalha num grafo. A equação da difusão de calor é:

$$\frac{\partial x}{\partial t} = -Lx$$

- **Intuição:** A informação (x) flui dos nós com valores altos para os seus vizinhos com valores baixos, a uma taxa proporcional à diferença (exatamente o que L mede).
- Processos iterativos como o **PageRank** são uma forma de difusão.

Difusão e Cascades em Grafos III

Modelos de Cascade (ex: Independent Cascade Model)

Modelos de epidemia ou influência (ex: como um rumor se espalha).

- **Iteração k :** Um conjunto de nós V_{ativo} está ativo.
- **Iteração $k + 1$:** Cada nó $u \in V_{ativo}$ tem uma probabilidade p_{uv} de ativar os seus vizinhos v inativos.

O Padrão Comum

Tanto a difusão (Laplaciano) como as cascades (modelos de influência) são processos **iterativos** onde o estado de um nó em $t + 1$ é determinado pela **agregação dos seus vizinhos** em t .

Problemas Típicos em Grafos

O que queremos prever?

- **Classificação de Nós (Node Classification):**

- *Ex: Este utilizador (nó) numa rede social é um bot ou um humano?*

Problemas Típicos em Grafos

O que queremos prever?

- **Classificação de Nós (Node Classification):**

- *Ex: Este utilizador (nó) numa rede social é um bot ou um humano?*

- **Classificação de Grafos (Graph Classification):**

- *Ex: Esta molécula (grafo) é tóxica ou não?*

Problemas Típicos em Grafos

O que queremos prever?

- **Classificação de Nós (Node Classification):**

- *Ex: Este utilizador (nó) numa rede social é um bot ou um humano?*

- **Classificação de Grafos (Graph Classification):**

- *Ex: Esta molécula (grafo) é tóxica ou não?*

- **Previsão de Arestas (Link Prediction):**

- *Ex: Devemos recomendar a amizade entre o utilizador A e B?*

Problemas Típicos em Grafos

O que queremos prever?

- **Classificação de Nós (Node Classification):**

- *Ex: Este utilizador (nó) numa rede social é um bot ou um humano?*

- **Classificação de Grafos (Graph Classification):**

- *Ex: Esta molécula (grafo) é tóxica ou não?*

- **Previsão de Arestas (Link Prediction):**

- *Ex: Devemos recomendar a amizade entre o utilizador A e B?*

- **Clustering de Nós (Node Clustering):**

- *Ex: Encontrar grupos (comunidades) de utilizadores com interesses semelhantes.*

Problemas Clássicos em Grafos

O objetivo destes algoritmos de difusão é resolver tarefas fundamentais:

- **Importância de Nós (PageRank, HITS):**
 - Encontrar os nós mais centrais ou influentes num grafo.
 - Ex: Quais as páginas web mais importantes?

Problemas Clássicos em Grafos

O objetivo destes algoritmos de difusão é resolver tarefas fundamentais:

- **Importância de Nós (PageRank, HITS):**
 - Encontrar os nós mais centrais ou influentes num grafo.
 - Ex: Quais as páginas web mais importantes?
- **Classificação de Nós (Node Classification):**
 - Prever uma etiqueta para cada nó, assumindo que vizinhos partilham etiquetas (homofilia).
 - Ex: Este utilizador (nó) é um bot ou um humano?

Problemas Clássicos em Grafos

O objetivo destes algoritmos de difusão é resolver tarefas fundamentais:

- **Importância de Nós (PageRank, HITS):**
 - Encontrar os nós mais centrais ou influentes num grafo.
 - Ex: Quais as páginas web mais importantes?
- **Classificação de Nós (Node Classification):**
 - Prever uma etiqueta para cada nó, assumindo que vizinhos partilham etiquetas (homofilia).
 - Ex: Este utilizador (nó) é um bot ou um humano?
- **Clustering de Nós (Node Clustering / Community Detection):**
 - Encontrar grupos de nós que estão mais densamente ligados entre si do que com o resto do grafo.
 - Ex: Encontrar comunidades de utilizadores com interesses semelhantes.

Todos estes métodos dependem da **agregação de vizinhança**.

Algoritmos Clássicos: PageRank I

PageRank

Usado pela Google para classificar a importância de páginas web.

Ideia: A importância de um nó (r_i) é a soma da importância dos nós (r_j) que apontam para ele, normalizada pelo número de links de saída desses nós (D_j).

■ Processo Iterativo:

$$r_i^{(k+1)} = \sum_{j \rightarrow i} \frac{r_j^{(k)}}{D_{jj}}$$

■ Em forma matricial (com *damping factor* α):

$$\mathbf{r}^{(k+1)} = \alpha(\mathbf{A}^T \mathbf{D}^{-1}) \mathbf{r}^{(k)} + (1 - \alpha) \frac{1}{N} \mathbf{1}$$

Algoritmos Clássicos: HITS I

Hubs and Authorities (Kleinberg 1999)

Cada nó i tem duas pontuações que se reforçam mutuamente:

- **Authority** (a_i): Um nó que contém informação valiosa (ex: o artigo original).
- **Hub** (h_i): Um nó que aponta para muitas boas Autoridades (ex: um artigo de review).

Algoritmos Clássicos: HITS II

Atualização Iterativa

- Uma boa Autoridade é apontada por bons Hubs:

$$a_i^{(k+1)} = \sum_{j \rightarrow i} h_j^{(k)} \quad \rightarrow \quad a \leftarrow A^T h$$

- Um bom Hub aponta para boas Autoridades:

$$h_i^{(k+1)} = \sum_{i \rightarrow j} a_j^{(k)} \quad \rightarrow \quad h \leftarrow A a$$

A Ideia Comum: Agregar Vizinhos I

Tanto o PageRank como o HITS são algoritmos **iterativos** onde o estado de um nó na iteração $k + 1$ é calculado **agregando** os estados dos seus vizinhos da iteração k .

A Semente da GNN

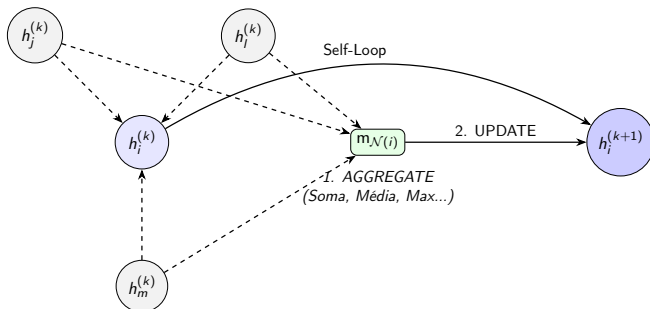
E se, em vez de uma fórmula *fixa* (como $A^T D^{-1}$), pudéssemos **aprender** a melhor forma de agregar e atualizar a informação?

Esta é a ideia central das **Graph Neural Networks (GNNs)**, formalizada como **Message Passing Neural Networks (MPNN)** (Gilmer et al. 2017).

O Framework de Message Passing

O Forward Pass de uma Camada GNN

Em cada camada k , para cada nó i , atualizamos o seu vetor de features (estado oculto) $h_i^{(k)}$ em dois passos:



Message Passing: A Matemática I

Passo 1: AGGREGATE (Função de Agregação)

Cada nó i recolhe os vetores (mensagens) $h_j^{(k)}$ dos seus vizinhos $j \in \mathcal{N}(i)$ e agrega-os num único vetor $m_{\mathcal{N}(i)}$.

Para garantir a **Invariância à Permutação** (a ordem dos vizinhos não importa), esta função AGG tem de ser uma operação simétrica:

$$m_{\mathcal{N}(i)}^{(k+1)} = \text{AGG} \left(\{ \phi(h_j^{(k)}) : j \in \mathcal{N}(i) \} \right)$$

(Onde ϕ é muitas vezes um MLP, como $\phi(h_j) = W_{\text{neigh}} h_j^{(k)}$)

Funções AGG Comuns: $\sum$ (Soma), $\text{mean}()$ (Média), $\text{max}()$.

Message Passing: A Matemática II

Passo 2: UPDATE (Função de Atualização)

O nó i combina a mensagem agregada dos seus vizinhos ($m_{\mathcal{N}(i)}$) com a sua **própria representação** da camada anterior ($h_i^{(k)}$) para calcular o seu novo estado, $h_i^{(k+1)}$.

$$h_i^{(k+1)} = \text{UPDATE} \left(\psi(h_i^{(k)}), m_{\mathcal{N}(i)}^{(k+1)} \right)$$

(Onde ψ é outro MLP, como $\psi(h_i) = W_{\text{self}} h_i^{(k)}$)

Uma GNN **aprende** os pesos das funções ϕ e ψ (os MLPs) através de retropropagação.

Algoritmos Clássicos como GNNs I

GNNs = Algoritmos de Difusão Generalizados

Podemos reescrever os algoritmos clássicos de difusão (PageRank, HITS) usando a linguagem das GNNs (AGGREGATE e UPDATE). Nesta visão, PageRank e HITS são GNNs onde as funções de agregação e atualização são **fixas** e **não-aprendíveis**.

PageRank como uma GNN I

A Fórmula do PageRank (simplificada)

$$r_i^{(k+1)} = \sum_{j \in \mathcal{N}_{in}(i)} \frac{r_j^{(k)}}{D_{jj}}$$

Onde $r_i^{(k)}$ é a pontuação do nó i na iteração k , $\mathcal{N}_{in}(i)$ são os vizinhos que apontam para i , e D_{jj} é o grau de saída (n.º de links) do nó j .

PageRank como uma GNN II

PageRank no Framework de GNN

Deixamos o estado do nó ser a sua pontuação: $h_i^{(k)} = r_i^{(k)}$.

- MESSAGE ϕ (do nó j para i):

$$m_{j \rightarrow i} = \frac{h_j^{(k)}}{D_{jj}}$$

- AGGREGATE (Soma):

$$m_i = \sum_{j \in \mathcal{N}_{in}(i)} m_{j \rightarrow i}$$

- UPDATE (Identidade):

$$h_i^{(k+1)} = m_i$$

HITS como uma GNN I

As Fórmulas do HITS

O HITS é mais complexo; cada nó i tem duas features: Authority (a_i) e Hub (h_i). O estado do nó é $h_i = [a_i, h_i]$. As atualizações são mutuamente recursivas:

$$a_i^{(k+1)} = \sum_{j \in \mathcal{N}_{in}(i)} h_j^{(k)} \quad \text{e} \quad h_i^{(k+1)} = \sum_{j \in \mathcal{N}_{out}(i)} a_j^{(k)}$$

HITS como uma GNN II

HITS no Framework de GNN (Atualização Vetorial)

Definimos o estado de cada nó como um vetor $h_i^{(k)} = [a_i^{(k)}, h_i^{(k)}]^\top$.
A atualização ocorre num único passo:

$$h_i^{(k+1)} = \sum_{j \in \mathcal{N}_{in}(i)} W_{in} h_j^{(k)} + \sum_{j \in \mathcal{N}_{out}(i)} W_{out} h_j^{(k)}$$

- **Agregação de Entrada (W_{in}):** Seleciona a componente h_j (Hub) dos vizinhos que apontam para i para atualizar a_i (Authority).
- **Agregação de Saída (W_{out}):** Seleciona a componente a_j (Authority) dos vizinhos apontados por i para atualizar h_i (Hub).

Variações de Arquitetura: GCN I

Diferentes arquiteturas de GNN (GCN, GraphSAGE, GAT) distinguem-se principalmente pela escolha das funções **AGG** e **UPDATE**.

1. GCN: Graph Convolutional Network (Kipf e Welling 2017)

A GCN utiliza uma média ponderada fixa baseada no grau dos nós (normalização espectral).

■ AGGREGATE (Soma Ponderada Fixa):

$$m_i = \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{d_i d_j}} h_j^{(k)}$$

(Os pesos dependem apenas da estrutura do grafo, não das features).

Variações de Arquitetura: GCN II

■ UPDATE (Linear + Ativação):

$$h_i^{(k+1)} = \text{ReLU}(W \cdot m_i)$$

Variações de Arquitetura: GraphSAGE I

2. GraphSAGE: SAmple & aggreGatE (Hamilton2017)

Uma arquitetura desenhada para escalabilidade (indutiva), que separa explicitamente a agregação da atualização.

■ AGGREGATE (Flexível):

$$m_i = \text{AGG}_{\text{sage}} \left(\{h_j^{(k)}, \forall j \in \mathcal{N}(i)\} \right)$$

Opções comuns: Mean (Média), Max-Pooling ou até uma LSTM (aplicada a uma permutação aleatória dos vizinhos).

Variações de Arquitetura: GraphSAGE II

- **UPDATE (Concatenação):** Ao contrário da GCN (que soma), o GraphSAGE **concatena** o estado anterior com a mensagem agregada antes da projeção.

$$h_i^{(k+1)} = \sigma \left(W \cdot [h_i^{(k)} \parallel m_i] \right)$$

Variações de Arquitetura: GAT I

3. GAT: Graph Attention Network (Velickovic2018)

Introduz o mecanismo de atenção para ponderar a importância dos vizinhos (anisotropia).

■ AGGREGATE (Soma Ponderada Apreendida):

$$m_i = \sum_{j \in \mathcal{N}(i)} \alpha_{ij} W h_j^{(k)}$$

Os pesos α_{ij} não são fixos (como na GCN), mas calculados por um mecanismo de atenção:

$$\alpha_{ij} = \text{softmax}_j \left(\text{LeakyReLU}(\vec{a}^T [W h_i \parallel W h_j]) \right)$$

Variações de Arquitetura: GAT II

■ UPDATE:

$$h_i^{(k+1)} = \sigma(m_i)$$

Conexão com Transformers

A GAT é conceptualmente idêntica a um Transformer, mas a atenção é mascarada pela matriz de adjacência (só atendemos aos vizinhos reais, não a todos os nós).

Variações de Arquitetura: GIN I

4. GIN: Graph Isomorphism Network (Xu et al. 2019)

Desenhada teoricamente para máxima expressividade (distinguir grafos não isomorfos).

- **AGGREGATE (Soma):** A GIN usa a **Soma** em vez da Média ou Max.

$$m_i = \sum_{j \in \mathcal{N}(i)} h_j^{(k)}$$

(A soma captura o tamanho da vizinhança, ao contrário da média, o que é crucial para distinguir estruturas químicas).

- **UPDATE (MLP):**

$$h_i^{(k+1)} = \text{MLP} \left((1 + \epsilon) h_i^{(k)} + m_i \right)$$

Resumo das Arquiteturas

Modelo	Agregação (AGG)	Combinação (UPDATE)	Pesos	Foco
GCN	Média Ponderada (Grau)	Soma	Fixos (Topologia)	Semi-Sup. / Eficiência
GraphSAGE	Média / Max / LSTM	Concatenação	Aprendidos	Indutivo / Grandes Grafos
GAT	Soma Ponderada (Atenção)	Soma	Dinâmicos (h_i, h_j)	Importância de Vizinhos
GIN	Soma Simples	Soma + MLP	Fixos (1)	Expressividade (WL-Test)

Tabela: Comparação das principais arquiteturas de Message Passing.

Quão Poderosas são as GNNs? I

O Problema do Isomorfismo de Grafos

Se dois grafos G_1 e G_2 são estruturalmente diferentes, podemos garantir que a nossa GNN irá produzir representações diferentes para eles?

Quão Poderosas são as GNNs? II

O Teste de Weisfeiler-Lehman (1-WL)

Um algoritmo clássico (e rápido) de color refinement para testar se dois grafos *não* são isomorfos.

- 1 $k=0$:** Todos os nós recebem a mesma cor (ou a sua feature inicial).
- 2 $k \rightarrow k+1$:** Cada nó i atualiza a sua cor:

$$c_i^{(k+1)} = \text{HASH} \left(c_i^{(k)}, \text{MULTICONJUNTO}(\{c_j^{(k)} \mid j \in \mathcal{N}(i)\}) \right)$$

- 3 Parar:** Se os histogramas de cores dos dois grafos forem diferentes, eles são **não-isomorfos**. Se forem iguais, o teste é **inconclusivo**.

O Limite das GNNs de Message Passing I

Teorema (Morris et al., 2019; Xu et al., 2019)

As GNNs baseadas em Message Passing (como GCN, GIN, GraphSAGE) são, no máximo, **tão poderosas quanto o teste 1-WL**.

Porquê?

A fórmula de atualização da GNN é funcionalmente idêntica à atualização de cores do 1-WL:

$$h_i^{(k+1)} = \text{UPDATE} \left(h_i^{(k)}, \text{AGG}(\{h_j^{(k)} \mid j \in \mathcal{N}(i)\}) \right)$$

O Limite das GNNs de Message Passing II

- Isto significa que qualquer par de grafos que o 1-WL não consegue distinguir (ex: duas moléculas não-isomorficas que são k -regulares), uma GNN de message passing também **não conseguirá distinguir**.
- A **Hierarquia WL** (k -WL) define testes mais poderosos, que inspiram GNNs mais expressivas (mas computacionalmente mais caras).

A Grande Ligação I

Um Transformer é (funcionalmente) um tipo de GNN.

O Transformer opera num Grafo...

...um **grafo completo** (fully connected graph).

- Numa frase com N tokens, o Transformer assume que cada token está ligado a todos os outros $N - 1$ tokens.
- Os nós são os tokens (ex: O, gato, sentou).
- As arestas são as linhas de atenção.

A Grande Ligação II

A Auto-Atenção é Message Passing

- **GNN (GCN):** Os pesos das arestas são **estáticos**, definidos pela estrutura do grafo (a matriz A).
- **Transformer (Atenção):** Os pesos das arestas são **dinâmicos**, calculados a cada passagem.

GNN vs. Transformer (Agregação) I

Vamos comparar o passo de AGREGAÇÃO:

GNN (GCN)

$$m_i = \sum_{j \in \mathcal{N}(i) \cup \{i\}} \underbrace{\frac{1}{\sqrt{d_i d_j}}}_{\text{Peso Fixo}} \cdot \underbrace{h_j}_{\text{Mensagem}}$$

- A topologia ($\mathcal{N}(i)$) é **dada** (esparsa).
- Os pesos são **fixos** (baseados na estrutura).

GNN vs. Transformer (Agregação) II

Transformer (Self-Attention)

$$m_i = \sum_{j=1}^N \underbrace{\alpha_{ij}}_{\text{Peso Dinâmico}} \cdot \underbrace{V_j}_{\text{Mensagem}}$$

Onde:

$$\alpha_{ij} = \text{softmax} \left(\frac{Q_i K_j^T}{\sqrt{d_k}} \right)$$

- A topologia é **trivial** (grafo completo).
- Os pesos são **aprendidos** (baseados no conteúdo Q, K).

GNN vs. Transformer (Agregação) III

Conclusão

Um Transformer aprende a *estrutura do grafo* (sintaxe/semântica) que a GNN recebe como entrada, tratando a sequência como um grafo completo.

Referências I

-  Gilmer, Justin et al. (2017). *Neural Message Passing for Quantum Chemistry*. arXiv: 1704.01212 [cs.LG]. Framework de Message Passing (MPNN).
-  Kipf, Thomas N. e Max Welling (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. arXiv: 1609.02907 [cs.LG].
-  Kleinberg, Jon M (1999). “Authoritative sources in a hyperlinked environment”. Em: *Journal of the ACM (JACM)* 46.5, pp. 604–632. Algoritmo HITS.
-  Xu, Keyulu et al. (2019). *How Powerful are Graph Neural Networks?* arXiv: 1810.00826 [cs.LG]. Conexão GNN e Teste WL (GIN).